GUIA TÉCNICO | 06

Backpropagation, MLP e problema XOR

Capítulo 06/20 · Fundamentos, experiência reproduzível e interpretação

PROPÓSITO E OBJETIVOS

- Aplicar regra da cadeia e mostrar por que uma camada oculta representa XOR.
- Relacionar a técnica à sua função de perda, representação e protocolo de avaliação.
- Explicar limitações e formular um teste reprodutível.

MAPA DE ESTUDO

1. Fundamentação do método
2. Expressões e hipóteses
3. Experimento essencial executável
4. Laboratório aplicado e limitações
5. Exercício e verificações

01 FUNDAMENTAÇÃO CONCEITUAL

Retropropagação propaga derivadas da perda da saída para as camadas iniciais. É um algoritmo para calcular gradientes; o otimizador decide como atualizar parâmetros. Ativações não lineares são necessárias para representar relações como XOR.

Fonte histórica de apoio: notas do curso original, consultadas apenas como referência de contexto; as explicações e exemplos desta edição são independentes.

Questões críticas: quais pressupostos são necessários? O que a função aprende? Qual métrica indica utilidade real? Que tipo de vazamento ou má interpretação invalidaria a conclusão?

02 MATEMÁTICA E DIMENSÕES

Expressão esquemática: δ _l=(W_(l+1) δ _(l+1)) ⊙ f′(z_l)

A expressão acima resume a relação central. Para a derivação, identifique dimensões, hipóteses e operações; não interprete notação compacta como uma implementação completa.

LEITURA CRÍTICA DO CAPÍTULO

A dedução completa exige derivadas de cada camada, propagação reversa e atualização distinta dos pesos. A versão NumPy explicita todas essas etapas.

Procedimento: execute a próxima célula com o núcleo declarado em requirements.txt, confira os assert e interprete os resultados. O experimento é independente dos exemplos anteriores.

03 EXPERIMENTO ESSENCIAL

Código extraído do notebook revisado. Execute a célula completa no JupyterLab; as quebras visuais abaixo não substituem o arquivo executável.

```
import numpy as np
X=np.array([[0,0],[0,1],[1,0],[1,1]],float)
# Representação explícita por duas unidades ocultas: OR e NAND
h1=(X.sum(axis=1)>=1).astype(int)
h2=(X.sum(axis=1)<2).astype(int)
yhat=(h1+h2==2).astype(int)
print('OR:',h1.tolist(),'NAND:',h2.tolist(),'XOR:',yhat.tolist())
assert np.array_equal(yhat,[0,1,1,0])
```

INTERPRETAÇÃO

Compare o resultado obtido com a formulação matemática. Confirme shapes, escala e hipótese de modelagem. Um experimento sintético valida uma operação, não substitui avaliação em dados

reais. Ao treinar modelos, separe dados e ajuste transformações apenas no treino; preserve um conjunto final de teste.

04 LABORATÓRIO APLICADO

Laboratório aplicado revisado — Retropropagação numérica em uma MLP para XOR

A dedução completa exige derivadas de cada camada, propagação reversa e atualização distinta dos pesos. A versão NumPy explicita todas essas etapas.

Procedimento: execute a próxima célula com o núcleo declarado em requirements.txt, confira os assert e interprete os resultados. O experimento é independente dos exemplos anteriores.

```
import numpy as np
X = np.array([[0,0],[0,1],[1,0],[1,1]], dtype=float)
y = np.array([[0],[1],[1],[0]], dtype=float)
rng = np.random.default_rng(17)
W1 = rng.normal(0, 0.7, (2, 8)); b1 = np.zeros((1,8))
W2 = rng.normal(0, 0.7, (8, 1)); b2 = np.zeros((1,1))
def sig(z): return 1/(1+np.exp(-np.clip(z,-30,30)))
losses=[]
for epoch in range(3000):
    a1 = np.tanh(X @ W1 + b1)
    p = sig(a1 @ W2 + b2)
    loss = -np.mean(y*np.log(p+1e-9)+(1-y)*np.log(1-p+1e-9))
    losses.append(loss)
    d2=(p-y)/len(X)
    dW2=a1.T@d2; db2=d2.sum(axis=0,keepdims=True)
    d1=(d2@W2.T)*(1-a1**2)
    dW1=X.T@d1; db1=d1.sum(axis=0,keepdims=True)
    W2-=0.8*dW2; b2-=0.8*db2; W1-=0.8*dW1; b1-=0.8*db1
pred=(sig(np.tanh(X@W1+b1)@W2+b2)>=.5).astype(int).ravel()
assert np.array_equal(pred, y.ravel().astype(int)), pred
assert losses[-1]<losses[0]
print('Predições XOR:',pred.tolist(),'perda ↪
    inicial/final:',round(losses[0],4),round(losses[-1],4))
```

Treino na própria tabela-verdade demonstra capacidade de ajuste, não generalização. Verificar gradientes e inicialização; XOR tem somente quatro casos.

Checklist: conferir entradas e saídas, sementes, inexistência de uso dos dados de teste durante ajuste e distinção entre uma demonstração sintética e desempenho em aplicações reais.

05 EXERCÍCIO E SOLUÇÃO COMENTADA

Por que ativação puramente linear em todas as camadas não resolve XOR?

Resposta: A composição de funções afins permanece afim; não cria fronteiras não lineares.

LIMITAÇÕES E BOAS PRÁTICAS

- Confundir ajuste no treino com generalização para dados não vistos.
- Reportar somente uma métrica ou uma execução aleatória.
- Ignorar shape, tipo de variável e convenção do backend.
- Executar exemplos com pesos baixados sem registrar versão, licença e procedência.
- Confundir demonstração matemática com validação de um modelo de produção.

VERIFICAÇÃO LOCAL

Execute Reiniciar kernel e executar todas as células. Registre versões do Python e das dependências, confira shapes e métricas e diferencie os laboratórios opcionais dos exemplos NumPy/scikit-learn.

Validação posterior (13/11/2026): notebooks 14 a 20 executados com TensorFlow CPU; opcionais 01 a 13 não integralmente revalidados.

REFERÊNCIAS TÉCNICAS

NumPy: numpy.org/doc · scikit-learn: scikit-learn.org/stable/user_guide.html · Keras: keras.io/guides · TensorFlow: tensorflow.org/guide